

Data Acquisition software for coincidence Gamma-Gamma
spectrometry

Project report

by

Akshar Ratanpara
(Roll No. IU2141230238)

Under the guidance of

Mr. Pranav R. Adhyaru (External Guide) & Ms. Madhvi A. Bera (Internal Guide)



Geosciences Division

Physical Research Laboratory, Ahmedabad, India.

submitted to



Department of Computer Science

Indus Institute of Technology Engineering, Ahmedabad, India.

2021 - 2025

A project report entitled

Data Acquisition software for coincidence Gamma-Gamma spectrometry

by

Akshar Ratanpara

(Roll No. IU2141230238)



**Akshar Ratanpara
(Student)**



**Pranav R. Adhyaru
(Project supervisor)**

Thesis printed on: 24/05/2025

Place: PRL, Ahmedabad

Acknowledgements

I would like to express my heartfelt gratitude to everyone who supported and guided me throughout the duration of my Final Semester Project at the **Geosciences Division, Physical Research Laboratory (PRL), Ahmedabad**.

First and foremost, I sincerely thank **Mr. Pranav R. Adhyaru**, my project supervisor, for his constant mentorship, technical guidance, and insightful suggestions throughout the development of this project. His encouragement and constructive feedback were instrumental in shaping both the direction and quality of the work.

I am also thankful to the entire PRL team for fostering a professional and collaborative research environment. Their support and access to valuable resources significantly enhanced my learning and enabled me to apply theoretical knowledge to a real-world scientific application.

I would also like to extend my sincere thanks to **Mr. Lakhansinh G. Chavda** for his support and guidance during the project. His encouragement and help were greatly appreciated and contributed meaningfully to my experience.

This project experience has been a defining part of my academic journey, allowing me to work at the intersection of software engineering and experimental nuclear physics. I am truly grateful for the opportunity to contribute to this important research initiative.

Akshar Ratanpara

Table of content

1. Introduction	5
2. Real-time generation and visualization of Coincidence Gamma-Gamma Spectrom	7
Overview	7
Motivation and Necessity	7
Theoretical Background	7
System Architecture and Implementation	8
Features and Functionalities	9
Experimental Integration	9
3. Embedded Signal Capture and Digital Conversion Interface for Gamma Detection	11
4. System Interaction and Software Integration	13
5. Project Workflow	16
6. Alternate Implementation Approach	18
7. Conclusion and future outlook	20
8. Code Flow Explanation and Functional Breakdown	22
9. Codes	25

1. Introduction

Modern nuclear physics and gamma-ray spectroscopy heavily rely on real-time data acquisition systems for the analysis of coincident gamma events. Coincidence measurements provide enhanced resolution and help isolate specific nuclear transitions by identifying correlated photon pairs. To capture and analyze such coincident gamma-gamma events efficiently, robust software-hardware integration is essential.

This project presents a full-stack software system for **real-time data acquisition, visualization, and storage** of coincidence gamma-gamma spectra. Developed during my research internship at the **Geosciences Division of Physical Research Laboratory (PRL)**, the system is designed to interface with a gamma detection setup using Raspberry Pi GPIOs and process energy information in a two-dimensional coincidence matrix (Z-matrix). The Z-matrix captures occurrences where detector 1 and detector 2 simultaneously detect energy events.

The primary goal was to eliminate manual data processing and replace it with a **high-performance digital pipeline** capable of live visualization, user control, and automated communication. For this, two major technologies were integrated:

1. **C++ for low-level data acquisition**, GPIO interfacing, and shared memory streaming
2. **Python (Dash framework)** for interactive web-based plotting, WebSocket-based command/control, and user interface.

Inspired by the concept of modular beam generation systems in computational optics, the DAQ system was architected to separate acquisition, communication, and visualization concerns. This design enables flexibility and expandability — making it suitable not just for coincidence spectra, but potentially for any 2D real-time experimental data.

The system incorporates features such as:

- **Shared memory streaming** from C++ to Python for fast data transfer
- **WebSocket bi-directional communication** for live command execution (start, stop, clear)
- **Simulation fallback** in case the hardware is unavailable
- **Live plotting using heatmaps and 3D scatter plots** with Volume-Under-Curve (VUC) insights
- **Interactive Dash UI** to control acquisition, adjust view modes, export CSV, and monitor acquisition status

This project builds on foundational skills in embedded systems, signal processing,

Python web frameworks, and multithreaded programming. It also mirrors real-world laboratory setups, where acquisition hardware, computation, and user interfaces must seamlessly integrate.

By creating an end-to-end data acquisition and visualization framework grounded in practical lab use, this work contributes to the modernization of nuclear research instrumentation and provides a learning model for future real-time scientific software systems.

2. Real-time generation and visualization of Coincidence Gamma-Gamma Spectrom

Overview

The detection and analysis of coincident gamma-ray events is a cornerstone in experimental nuclear physics and gamma spectroscopy. Coincidence techniques enable the identification of correlated photon pairs emitted during nuclear decay, thus enhancing peak-to-background ratios, isolating nuclear transitions, and revealing nuclear structure information with higher precision.

This project presents a complete, real-time software framework that enables the acquisition, processing, and interactive visualization of **coincidence gamma-gamma spectra** using a Raspberry Pi-based data acquisition system. The entire pipeline—from digital event capture via GPIO to live heatmap/3D rendering in a web-based interface—is implemented with performance and scalability in mind.

Motivation and Necessity

In traditional setups, coincidence spectra acquisition requires expensive proprietary software or cumbersome offline data processing pipelines. Many laboratory instruments capture pulse events but lack real-time, synchronized visualization or flexible control.

This project addresses these limitations by implementing:

- A high-throughput shared memory data stream from C++ to Python
- Real-time matrix updates of detected coincidences (Z-matrix)
- A Python Dash dashboard with interactive plotting, CSV export, and live control
- WebSocket-based full-duplex command and status communication

By merging low-level hardware interfacing with modern visualization tools, the system empowers researchers to **monitor nuclear event statistics live**, adjust acquisition in real time, and simulate data when hardware is disconnected—making it ideal for both active research and academic environments.

Theoretical Background

Coincidence gamma spectroscopy is based on detecting two gamma photons that are emitted simultaneously from a single nuclear event. These photons are recorded in two separate detectors, and only events that occur within a specific timing window are considered coincident.

Each coincidence event corresponds to a unique (E1, E2) pair—energies detected in Detector 1 and Detector 2, respectively. These pairs are accumulated into a 2D histogram $Z[E1][E2]$ known as the coincidence matrix.

In this system:

- E1 and E2 are derived from 13-bit parallel digital lines read via Raspberry Pi GPIO pins.
- Coincidence is triggered by a rising edge on a TRIGGER GPIO pin.
- Events are binned into a 8192x8192 matrix representing possible energy combinations.

The **Z-matrix** thus serves as a statistical map of correlated energy levels and is critical for identifying decay schemes, gamma cascades, or nuclear structure phenomena.

System Architecture and Implementation

Data Acquisition (C++)

- Uses pigpio library for GPIO interfacing on Raspberry Pi
- Reads 13-bit parallel data from two detectors (X and Y lines)
- Captures trigger events via an interrupt on a designated GPIO pin
- Updates a shared memory matrix `/zdata_shared` using `mmap()` and `shm_open()`
- Tracks pulse count and acquisition frequency
- Supports commands: START, STOP, CLEAR via stdin (soon to be WebSocket-based)

Live Visualization (Python)

- Built with Dash, Plotly, and NumPy
- Connects to the shared memory segment to read the `Z[][]` matrix
- Periodically refreshes plots using `dcc.Interval`
- Provides both 2D heatmaps and 3D scatter plots with VUC (Volume Under Curve) analysis
- Allows control via buttons (Start, Stop, Clear, Pause Plotting)
- Supports CSV export and simulation fallback (generates random Z-matrix)

Communication Layer

- Uses WebSocket protocol to send commands and receive acquisition statistics
- Enables real-time feedback (frequency, event count, errors)
- Supports dynamic control of acquisition from the Python UI

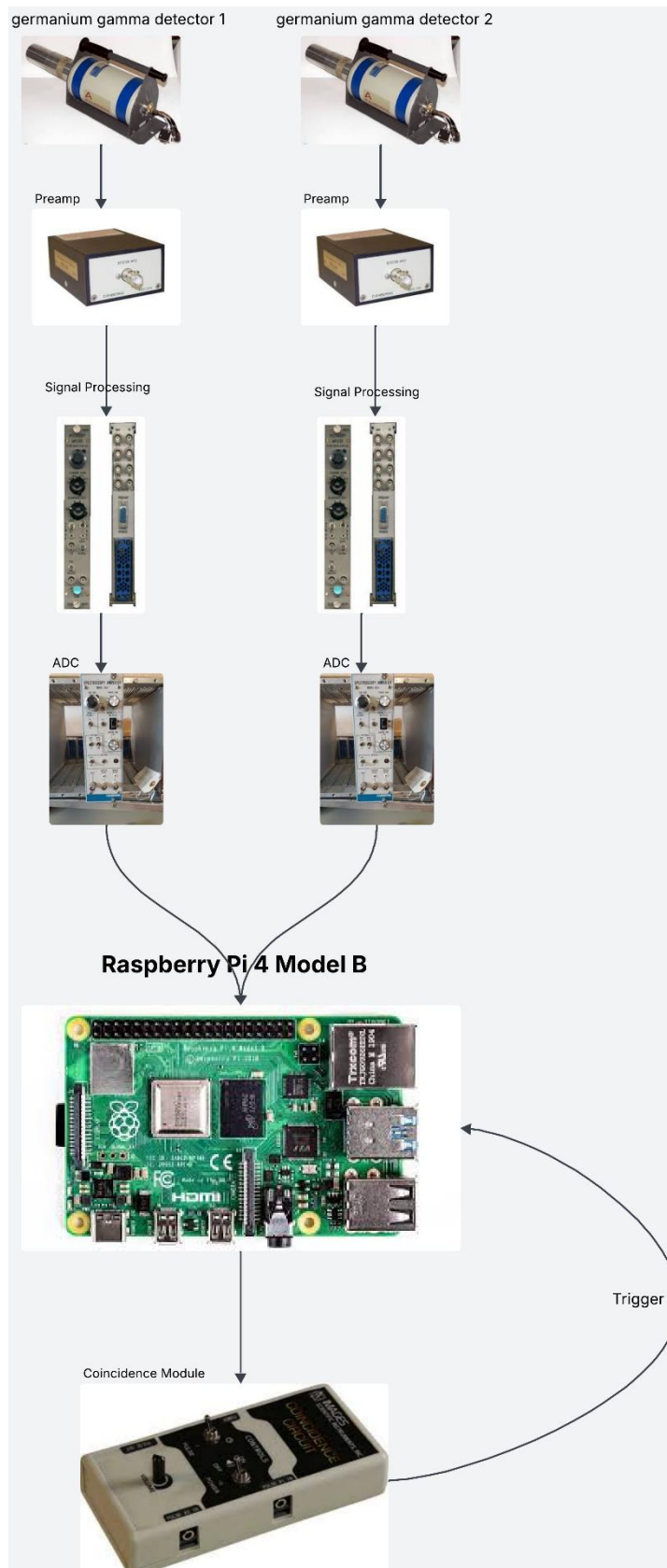
Features and Functionalities

- **Real-Time Acquisition:** GPIO events are processed with microsecond precision.
- **High-Resolution Coincidence Matrix:** Supports 8192x8192 bins using efficient memory mapping.
- **Shared Memory Architecture:** C++ writes directly to memory mapped segment; Python reads asynchronously.
- **WebSocket Control:** Enables remote or automated UI control over the DAQ.
- **Live Plotting Modes:** 3D VUC scatter plots and 2D heatmaps.
- **Plotting Pause Mode:** Pause rendering without halting acquisition.
- **Simulation Mode:** Automatically generates test data if hardware is not connected.
- **Exportable Data:** Users can save the Z-matrix as a timestamped CSV file.

Experimental Integration

The software was tested and deployed at the **Geosciences Division, PRL**, using a Raspberry Pi 4 interfaced with gamma-ray detectors. The digital outputs from detector electronics were connected to GPIO pins, with a multiplexer handling 13-bit decoding. Event triggers were reliably captured and processed.

Visualization was rendered in real time on a local network dashboard and monitored live by researchers for evaluating detector alignment and system performance.



3. Embedded Signal Capture and Digital Conversion Interface for Gamma Detection

Precise and high-frequency signal acquisition is vital in gamma spectroscopy systems, especially when detecting coincident events across multiple channels. This section details the embedded hardware setup developed for real-time interfacing between analog gamma detection equipment and a digital processing unit.

The hardware architecture consists of three primary components:

- **Two High-Purity Germanium (HPGe) Gamma Detectors**
- **Analog-to-Digital Converter (ADC)**
- **Raspberry Pi 4B (Data Acquisition Unit)**

These components work together to accurately capture photon detection events, convert the signals into digital form, and transfer the processed data to a software system for visualization and analysis.

Signal Processing Workflow

1. **Photon Detection:** Each HPGe detector captures gamma-ray events in real-time. These analog pulses contain energy information corresponding to the incident photon.
 2. **Analog-to-Digital Conversion:** The analog signals from both detectors are passed to the ADC, which digitizes the energy values. The ADC encodes the output as 13-bit digital data per channel, making it compatible with digital processing platforms.
 3. **Digital Input to Raspberry Pi:** The Raspberry Pi 4B is configured to read these 13-bit digital signals through its GPIO pins. For each coincident event, the Pi captures the two energy values (from Detector 1 and Detector 2) simultaneously.
 4. **Real-Time Encoding and Plotting:**
 - The GPIO inputs are interpreted as bit patterns.
 - Custom C++ code converts each 13-bit pattern into a decimal index representing energy level.
 - These indices are mapped into a 2D matrix ($Z[x][y]$), where each entry corresponds to a detected gamma-gamma event.
 - The matrix is continuously updated in shared memory and accessed by the Python dashboard for plotting.
-

Performance and Accuracy

The embedded system is optimized for **up to 60 kHz** signal processing frequency. This ensures that the system captures and records events without bottlenecks or data loss even in high-flux experimental environments. The GPIO polling and interrupt mechanisms are configured for microsecond-level responsiveness, balancing latency with throughput.

Key Features

- **Non-blocking GPIO Read:** Ensures consistent data flow even under load.
- **Real-Time Processing:** Minimal latency between physical event and digital visualization.
- **Modular Design:** Detectors, ADC, and Pi can be reconfigured or scaled for larger systems.
- **Code-Level Bit Decoding:** Mapping 13-bit digital lines into clean, integer energy values.

Applications and Integration

This embedded hardware setup was fully integrated into the larger Coincidence Gamma-Gamma DAQ framework developed during this project. The Pi handles the low-level event decoding, while the data is passed seamlessly to a web-based UI for researchers to analyze.

Such systems are ideal for:

- Nuclear physics experiments requiring real-time multi-detector synchronization
- Experimental setups with high event rates and low tolerance for data loss
- Educational environments showcasing digital interfacing with radiation sensors

In conclusion, this embedded control and acquisition subsystem acts as the backbone of the real-time gamma coincidence detection system, translating analog signals into structured, timestamped digital data for visualization, analysis, and archival.

4. System Interaction and Software Integration

A critical aspect of the Coincidence Gamma-Gamma Spectroscopy project is the seamless interaction between the acquisition hardware (C++ code) and the visualization interface (Python Dash app). This bi-directional communication architecture ensures high-throughput data transfer, responsive UI updates, and real-time control—all essential for experimental research.

The system follows a **hybrid communication strategy** involving:

- **Shared memory mapping** for high-speed Z-matrix updates
- **WebSocket protocol** for command and status exchange between Python and C++

Data Flow Architecture

1. Signal Detection and Processing (C++ Layer)

- C++ code running on the Raspberry Pi initializes GPIOs and shared memory (/zdata_shared).
- When a trigger pulse is detected, the system reads digital outputs from the ADC.
- These 13-bit inputs are decoded into two integers (x, y), which represent detected energy values.
- The corresponding entry $Z[x][y]$ in the shared memory matrix is incremented.

2. Memory Sharing for Real-Time Access

- A `mmap()` segment allows the Python code to read the C++-generated matrix.
- This enables rapid visualization without file I/O or delays.
- The matrix can grow large (e.g., 8192x8192), so Python applies downsampling when needed for display.

3. UI & Visualization (Python Dash)

- The Python Dash application reads shared memory periodically (1 Hz by default) and updates either a 2D heatmap or 3D VUC (Volume Under Curve) plot.
 - A control panel allows users to start, pause, clear, or resume data acquisition.
 - The app also tracks acquisition rate, event count, and status indicators such as "Simulating" or "Disconnected".
-

WebSocket Command and Control

To enable interactive control from the Python interface, a **WebSocket client-server model** was implemented:

- The **C++ process hosts a WebSocket server** (typically on port 8765)
- The **Python app acts as a WebSocket client**
- JSON messages are used for command dispatch and status updates

Commands sent from Python to C++ include:

- { "command": "START" } → Enables acquisition by setting the GPIO alert callback
- { "command": "STOP" } → Pauses data acquisition
- { "command": "CLEAR" } → Clears the shared Z-matrix using memset()

Status messages from C++ to Python include:

- { "frequency": 30000, "total_events": 19430 }
- { "error_type": "Hardware_Disconnected", "message": "DAQ hardware not initialized." }
-

The **Python Dash UI** listens for these updates in a non-blocking background thread and updates live labels accordingly.

Simulation Fallback

If the shared memory file is not detected (e.g., due to C++ crash or startup delay), the Python app enters **Simulation Mode**. In this mode:

- Randomized Z-matrix data is generated using NumPy
- UI still functions identically, allowing for UI testing and demonstration
- An alert badge is displayed stating: "Simulating (Random Data)"

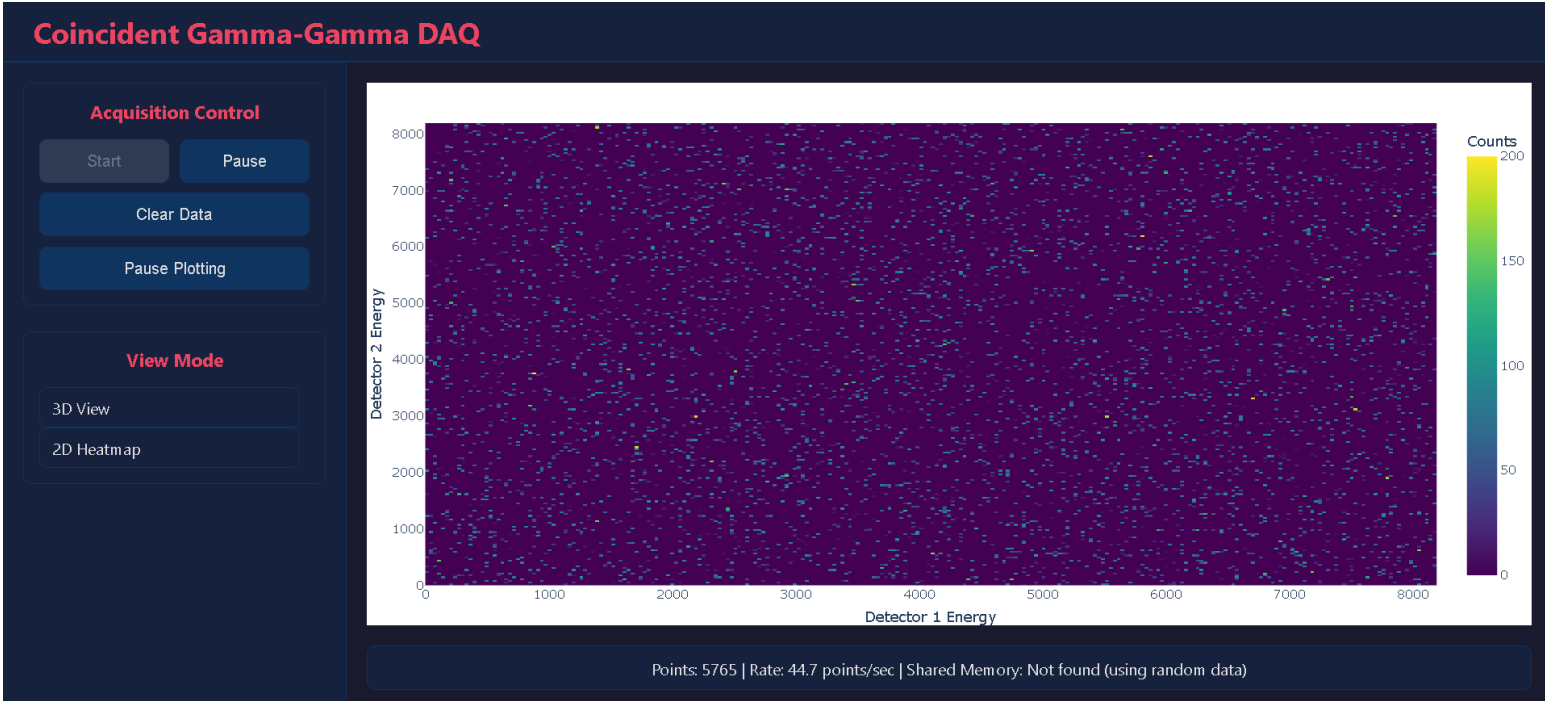
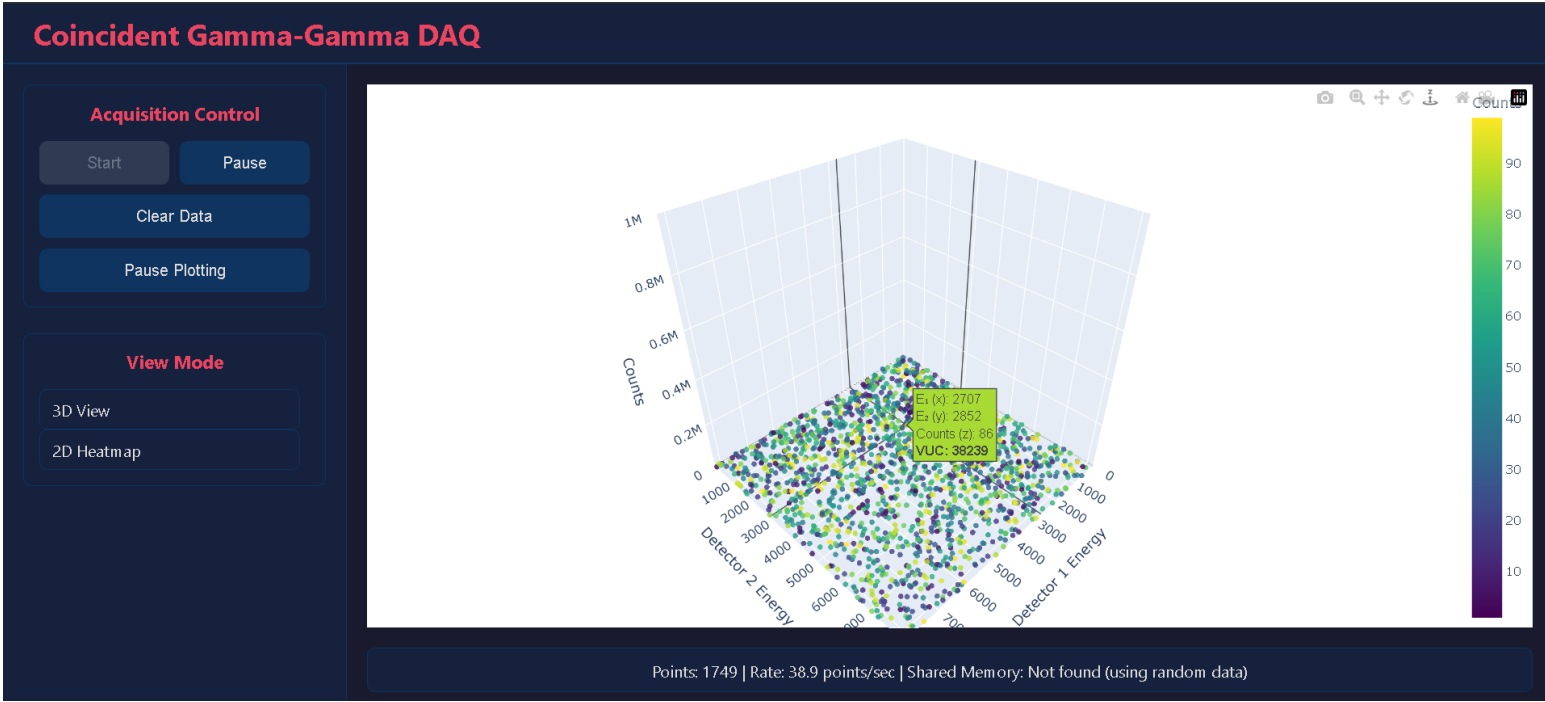
Synchronization and Cleanup

The system is designed with graceful startup and shutdown behavior:

- On launch, Python attempts to spawn the C++ process using subprocess.Popen()
- On exit (e.g., via CTRL+C), Python invokes terminate() and closes shared memory
- Both processes log critical errors (e.g., mmap failure, GPIO init errors)

This client-server interaction framework enables a robust and modular system where both high-speed data and low-latency controls are managed efficiently. By blending memory sharing for data throughput with WebSocket messaging for control, the system reflects real-world standards for embedded data acquisition networks and responsive scientific visualization environments.

GUI screen



5. Project Workflow

The project follows a modular and layered workflow designed to separate hardware data acquisition, memory management, software control, and visualization. Each stage communicates with the next through well-defined interfaces, allowing for maintainability, performance optimization, and real-time responsiveness.

Step-by-Step Workflow Overview:

1. Gamma Detection (Hardware Layer)

- Two High-Purity Germanium (HPGe) detectors capture gamma-ray events.
- Analog output pulses from the detectors represent the detected gamma energies.

2. Analog-to-Digital Conversion (ADC Interface)

- The analog pulses are fed into a high-speed ADC module.
- The ADC converts each pulse into 13-bit parallel digital signals per detector channel (X and Y).

3. Signal Capture via Raspberry Pi (Embedded Control Layer)

- The 13-bit digital outputs from the ADC are connected to Raspberry Pi GPIO pins.
- A custom C++ program on the Pi:
 - Initializes GPIO pins for input.
 - Sets up interrupt-based trigger detection.
 - Reads the 13-bit digital lines when a rising edge (trigger) is detected.
 - Converts these bits to decimal values representing energy bins.
 - Increments the corresponding $Z[x][y]$ entry in a 2D coincidence matrix.

4. Shared Memory Buffering (Inter-process Communication Layer)

- The C++ code creates a shared memory object (`/zdata_shared`) using `shm_open` and `mmap`.
- The $Z[x][y]$ matrix is written into shared memory in real time.

5. Python Dash UI (Visualization and Control Layer)

- The Dash app launches as a server-based dashboard on a browser interface.
- It connects to shared memory and reads the latest coincidence matrix at regular intervals.
- Based on user selection, the app renders:
 - A **2D heatmap** of energy correlations
 - A **3D VUC (Volume Under Curve)** scatter plot

6. WebSocket-Based Control Layer

- The Python app connects to the C++ process through WebSocket.
- UI buttons trigger WebSocket messages like START, STOP, CLEAR.
- The C++ program listens for these commands and performs corresponding actions.
- Status updates (frequency, event count, error messages) are sent back to Python.

7. Simulation Mode (Fallback Logic)

- If shared memory is not detected (e.g., C++ not running), the system switches to simulation mode.
- Randomly generated matrix data simulates gamma events for testing and visualization.

8. Data Export and Cleanup

- Users can export the current coincidence matrix to CSV with timestamps.
- On shutdown, shared memory is cleaned, subprocesses terminated, and resources closed.

6. Alternate Implementation Approach

During the early stages of development, multiple hardware-software configurations were explored to implement a system for real-time coincidence gamma-gamma data acquisition and visualization. These approaches differed in architecture, data throughput, visualization latency, and overall flexibility. Below is a detailed summary of each approach and its limitations, which ultimately led to the adoption of the current C++ and Python hybrid system on Raspberry Pi.

1. LabVIEW-Only Implementation Using NI PCI-6224

This initial approach utilized **National Instruments' PCI-6224** DAQ card with a fully graphical software pipeline implemented in **LabVIEW**.

- **Workflow:** Gamma signals → ADC → NI PCI-6224 → LabVIEW block diagram
- **Features:** Hardware-timed single-point acquisition, digital input handling, graphical 2D data plotting

Drawbacks:

- System failed to handle input frequencies beyond **10 kHz**
- Repeated **“Memory Full”** errors due to buffer overflow in LabVIEW
- Limited flexibility for custom data interaction (e.g., zooming, filtering, real-time matrix operations)
- Not scalable for long-duration or high-throughput experiments

2. Raspberry Pi + LabVIEW (Two-Stage Architecture)

This hybrid approach used **Raspberry Pi 4B** for raw data acquisition from detectors and ADC, while **LabVIEW** handled the visualization.

- **Workflow:** Gamma signals → ADC → Raspberry Pi → Save data → LabVIEW pulls and displays CSV
- **Mechanism:** Handshake between Pi and PC; LabVIEW reads large CSV file when triggered

Drawbacks:

- Large datasets (e.g., 500MB CSV) caused severe **latency in visualization**
- Handshaking between devices was **unreliable**; occasional Pi crashes during file access
- Delayed and non-interactive UI response in LabVIEW
- Real-time performance was compromised; unsuited for live experimental control

3. Final Approach: Full Raspberry Pi with C++ + Python Integration

This is the finalized architecture and the core of this report.

- **C++ handles:** GPIO access, real-time acquisition, trigger detection, and shared memory streaming (/zdata_shared)
- **Python Dash:** Visualizes data live via heatmaps and 3D scatter plots; provides user controls through WebSocket
- **Performance:** Supports acquisition rates up to **60 kHz** reliably with real-time responsiveness

Advantages:

- Modular and efficient; each layer independently handles acquisition, communication, and visualization
- Simulation mode fallback improves robustness
- UI is portable, fast, and rich with control options (start, stop, clear, save)

7. Conclusion and future outlook

This project at the **Geosciences Division of the Physical Research Laboratory (PRL)** successfully demonstrates the design and implementation of a complete software-hardware framework for **Coincidence Gamma-Gamma Spectrum acquisition, control, and visualization**.

The developed system seamlessly integrates:

- High-precision **signal acquisition** via Raspberry Pi GPIO interfacing
- **Real-time data processing** using a custom C++ pipeline with shared memory
- A responsive and intuitive **Python Dash UI** for interactive 2D/3D visualization
- **WebSocket-based control** for starting, stopping, and monitoring experiments
- Simulation fallback logic to ensure full functionality even without hardware

By combining modern software architecture with embedded signal decoding, the platform bridges the gap between nuclear detection hardware and user-centric scientific software. The system was tested and deployed in a real experimental environment at PRL, where it showed robustness, flexibility, and usability.

This work highlights a practical and accessible approach to modern nuclear spectroscopy using open-source tools and low-cost hardware, and it forms the foundation for more sophisticated real-time data systems.

Future Outlook

In the future, this system can be extended to include:

-  **Multichannel ADC support** for handling multiple detectors simultaneously
-  **Dynamic Region-of-Interest (ROI) selection** directly from the UI
-  **Closed-loop control integration** (e.g., adjusting detector settings based on real-time statistics)
-  **Deployment on FPGA or Jetson platforms** for enhanced performance
-  **Advanced statistical tools** for spectrum fitting and decay pattern analysis
-  **Remote access capability** for cloud-based monitoring and control
-  **Enhanced error diagnostics** and recovery mechanisms
-  **Integration with external timing modules** to correlate events across distributed setups

The modular nature of this system ensures it can evolve with emerging experimental needs. Whether applied in nuclear physics, radiation monitoring, or academic labs, it offers a reliable and extensible foundation for real-time spectrum analysis and scientific exploration.

8. Code Flow Explanation and Functional Breakdown

This section outlines the core logic and structure of the `data_acquisition.cxx` program to ensure that future interns or researchers can quickly understand and build upon this work. The C++ code serves as the heart of the real-time gamma detection system, interfacing with hardware (GPIO), performing high-frequency acquisition, and streaming data to shared memory.

Key Functional Components

`main()`

- Initializes GPIO using `gpioInitialise()`
- Sets the `gpioSetAlertFunc(TRIGGER_PIN, callback)` to link interrupt handler
- Opens or creates shared memory using `shm_open()` and `mmap()`
- Starts a background thread for WebSocket communication
- Enters an infinite loop using `pause()` to keep the process alive

`gpio_callback()`

- Called automatically when a rising edge is detected on the TRIGGER pin
- Reads 13-bit digital values from two sets of GPIO pins (X and Y channels)
- Converts binary inputs into decimal indices (x, y)
- Atomically updates the 2D coincidence matrix `Z[x][y]++`
- Updates the global `pulseCount` for tracking event frequency

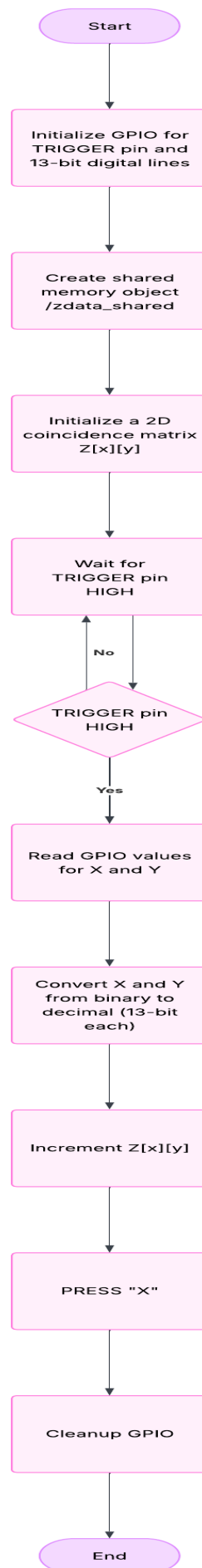
`websocket_thread()`

- Runs in parallel with the acquisition logic
- Listens for JSON commands (e.g., `{ "command": "START" }`)
- Sends real-time frequency and event statistics (e.g., `{ "frequency": 6000, "total_events": 24500 }`)
- Handles error conditions by sending structured error messages to Python

`clear_matrix()`

- Zeroes out the Z-matrix on receiving the CLEAR command
- Resets `pulseCount` and optionally `lastCount`

Flow Chart



Threading and Data Safety

- The acquisition thread and WebSocket thread run concurrently
- pulseCount is declared as `std::atomic<uint64_t>` to ensure thread safety
- `z[][]` matrix updates are kept lightweight to reduce interrupt callback latency

Shutdown and Resource Cleanup

- On process termination (e.g., SIGINT/Ctrl+C):
 - Shared memory is unmapped and unlinked
 - GPIOs are released using `gpioTerminate()`
 - WebSocket thread is gracefully terminated

Developer Tips and Notes

- **Pin Mapping:** GPIO pin numbers can be updated in the `#define` section at the top
- **Matrix Size:** Default matrix is 8192x8192; reduce this for faster tests
- **Testing Without Hardware:** Replace `gpioRead()` with random number generators to simulate input
- **Performance:** Avoid any `printf()` or logging inside `gpio_callback()`
- **Compile with:** `g++ -pthread -o data_acquisition data_acquisition.cxx -lpigpio -lrt -lpthread`

9. Codes

Link of github repository

<https://github.com/Akshar85/Coincident-Gamma-Gamma-Spectrum>